AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
I-CREDIT HOURS ENGINEERING PROGRAMS

Phase 2 report: Multiclass MNIST Classification with Advanced Model Improvements

CSE382: Introduction to Machine Learning

Submitted to:
- Dr. Hossam Hassan
- TA. Ali Ahmed

Team Members

1- Omar Ahmed Abouraia 23P0100
2- Hossam Osama Hussieny 23P0010
3- Mariam Ibrahim Assr 23P0123
4- Mariam Tarek Farouk 23P0214
5- Lujain Mohamed 23P0187

Contents

1. Introduction 3
2. Problem Definition 3
3. Dataset and Preprocessing 4
3.1 Dataset Overview 4
3.2 Data Preprocessing 4
4. Evaluation Metrics 5
5. Models Implemented 5
5.1 Softmax Regression 5
5.2 Linear SVM 6
5.3 Random Forest 6
5.4 CNN Feature Extraction 6
6. Feature Engineering 7
7. Hyperparameter Tuning 7
8. Regularization and Bias-Variance Analysis 9
9. Learning Curves 10
10. Models Compared 10
11. Model Comparison Discussion 11
12.Final Model Selection 12
13.Final Testing 12
14. Conclusion 13

1. Introduction

In Phase 1, the problem was restricted to binary classification between digits 3 and 8. In Phase 2, the problem is extended to a **multiclass classification task**, where the model must correctly classify digits from 0 to 9. This significantly increases the complexity of the problem.

The objective of this phase is not only to build accurate models, but also to **analyze and improve them using advanced techniques**, including:

- Hyperparameter tuning using cross-validation
- Regularization and bias–variance analysis
- Feature engineering using pretrained CNN
- Ensemble learning (Random Forest)
- Learning curve analysis

Multiple models were implemented from scratch, including Softmax Regression, Linear SVM, and Random Forest, and evaluated under different conditions.

2. Problem Definition

The problem is defined as a **supervised multiclass classification task**.

Mathematical Formulation:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

Where:

- $x_i \in \mathbb{R}^{784}$: flattened image
- $y_i \in \{0,1,2,3,4,5,6,7,8,9\}$: class label

Input:

- 28×28 grayscale image

Output:

$$y \in \{0,1,2,3,4,5,6,7,8,9\}$$

3. Dataset and Preprocessing

3.1 Dataset Overview

The MNIST dataset contains:

- 60,000 training images
- 10,000 testing images

Each image has dimensions $\mathbf{28} \times \mathbf{28}$, representing grayscale pixel intensities.

3.2 Data Preprocessing

Normalization

Pixel values are scaled to the range $[0, 1]$:

$$x' = \frac{x}{255}$$

This improves convergence during optimization.

Flattening

Each image is reshaped into a vector:

$$28 \times 28 \rightarrow 784$$

This allows classical machine learning models to process the data.

Train–Validation Split

The dataset is split into:

- Training set: 80%
- Validation set: 20%

Stratified sampling ensures equal class distribution.

Feature Scaling

Standardization is applied:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

This is essential for gradient-based methods and SVM.

4. Evaluation Metrics

To evaluate model performance, several metrics were used:

- **Accuracy**
- **Precision (macro)**
- **Recall (macro)**
- **F1-score (macro)**

Macro-averaging treats all classes equally, which is important for balanced evaluation. Confusion matrices were used to visualize misclassification patterns.

5. Models Implemented

5.1 Softmax Regression

Softmax Regression was implemented as the baseline model for this project due to its suitability for multi-class classification tasks. This model extends logistic regression by applying the softmax function to produce probability distributions across multiple classes. The baseline model was trained without regularization to establish an initial performance benchmark. The results from this model provided a reference point for further improvements.

Model:

$$\boldsymbol{z} = \boldsymbol{W}^{\boldsymbol{T}}\boldsymbol{x} + \boldsymbol{b}$$
$$\hat{\boldsymbol{y}}_{\boldsymbol{i}} = \frac{\boldsymbol{e}^{\boldsymbol{z}_{\boldsymbol{i}}}}{\sum_{\boldsymbol{j}} \boldsymbol{e}^{\boldsymbol{z}_{\boldsymbol{j}}}}$$

Loss Function:

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i) + \lambda \mid\mid W \mid\mid^2$$

Where:

- first term = cross-entropy loss
- second term = L2 regularization

Optimization
Gradient descent is used:

$$W := W - \alpha \nabla_W$$
$$b := b - \alpha \nabla_b$$

5.2 Linear SVM

A Linear Support Vector Machine (SVM) was implemented to explore an alternative classification approach. Unlike Softmax Regression, SVM focuses on maximizing the margin between classes using hinge loss. The model was trained and evaluated on the dataset, and its performance was compared with the tuned Softmax model. Results showed differences in how each model handled class boundaries.**Hinge .**

Loss:

$$L = \sum \max\,(0, s_j - s_{y_i} + 1)$$

This penalizes incorrect classifications.

5.3 Random Forest

The Random Forest model was implemented as an ensemble learning approach that combines multiple decision trees. Key hyperparameters such as the number of trees, maximum depth, and feature selection were tuned. Due to computational limitations, a subset of the dataset was used for training.**Key idea:**

- Train trees on random subsets
- Combine predictions using voting
- This improves robustness and reduces overfitting.

5.4 CNN Feature Extraction

To further improve performance, a hybrid approach was used where features were extracted using a pretrained Convolutional Neural Network (CNN), specifically

MobileNetV2. The images were resized and converted to RGB format before being processed by the CNN. The extracted features were then used as input for an SVM classifier. This approach leverages deep learning for feature extraction while maintaining a traditional classifier for prediction.

Pipeline:

$$\text{Image} \rightarrow \text{CNN} \rightarrow \text{Feature Vector} \rightarrow \text{SVM}$$

CNN significantly improves performance by:

- capturing edges
- detecting shapes
- reducing noise

6. Feature Engineering

Raw Features:

- Flattened pixel values
- Simple but limited representation

CNN Features:

- Extracted from pretrained CNN
- capturing complex patterns
- significantly improve performance

7. Hyperparameter Tuning

Cross-validation was used to systematically evaluate different hyperparameter combinations and select the best-performing model. A stratified k-fold approach ensured that each fold maintained a similar class distribution. Hyperparameters such as learning rate and regularization strength were tuned. This process provided a more reliable estimate of model performance compared to a single validation split.

Softmax:

- Learning Rate

- regularization strength

```
Iteration 0: Loss = 2.3027
```

	Learning Rate	L2 Regularization	Mean CV F1-score	Std CV F1-score
0	0.10	0.0010	0.898694	0.005492
1	0.10	0.0000	0.898534	0.005056
2	0.10	0.0001	0.898534	0.005056
3	0.05	0.0000	0.893858	0.004847
4	0.05	0.0001	0.893858	0.004847
5	0.05	0.0010	0.893329	0.005166

Best Configuration:

- Learning Rate = **0.1**
- L2 = **0.001**
- Mean CV F1 ≈ **0.8987**

Analysis

Softmax performs well but is limited by linear decision boundaries. It struggles to capture complex spatial relationships in image data.

SVM:

- Learning Rate
- regularization

	Learning Rate	Regularization	Mean CV F1-score	Std CV F1-score
0	0.0010	0.0001	0.882397	0.006217
1	0.0010	0.0100	0.881610	0.005268
2	0.0010	0.0010	0.881354	0.005023
3	0.0005	0.0010	0.874419	0.007565
4	0.0005	0.0100	0.874199	0.006571
5	0.0005	0.0001	0.873736	0.006717

Best Configuration:

- Learning Rate = 0.001
- Regularization = 0.0001

Analysis

SVM performs slightly worse than Softmax due to high-dimensional noisy input.

Compared Tuned Models

	Model	Feature Type	Learning Rate	Regularization	Accuracy	Precision	Recall	F1-score
0	Softmax Regression	Raw Pixels	0.100	0.0010	0.917250	0.916696	0.916277	0.916343
1	Linear SVM	Raw Pixels	0.001	0.0001	0.906667	0.906198	0.905553	0.905632

8. Regularization and Bias-Variance Analysis

To improve the baseline model, L2 regularization was introduced to reduce overfitting. Different regularization strengths were tested to observe their impact on training and validation performance. By comparing these results, it was possible to analyze the bias-variance tradeoff. A model with high variance tends to overfit the training data, while a model with high bias may underfit. The goal was to find a balance that optimizes generalization performance.

Observations:

- No regularization → high training accuracy, lower validation accuracy → overfitting
- Moderate regularization → balanced performance
- high regularization → underfitting

Softmax:

	Model	L2 Regularization	Train Accuracy	Validation Accuracy	Train-Val Gap	Diagnosis
0	Softmax Regression	0.0000	0.925750	0.916583	0.009167	Balanced
1	Softmax Regression	0.0001	0.925667	0.916500	0.009167	Balanced
2	Softmax Regression	0.0010	0.925083	0.916000	0.009083	Balanced
3	Softmax Regression	0.0100	0.918333	0.912417	0.005917	Balanced

Interpretation:

- Small gap → good generalization
- No overfitting
- Model is **balanced**

9. Learning Curves

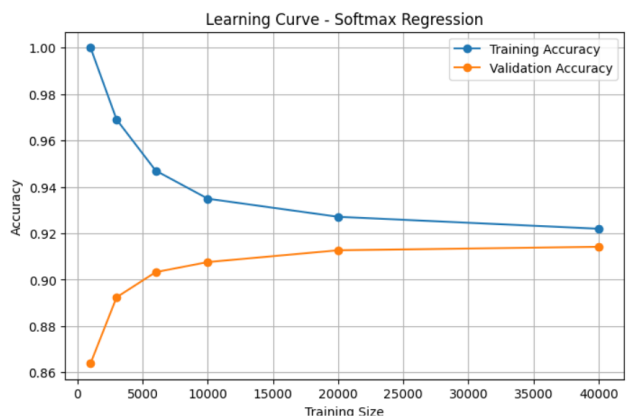
Learning curves show performance vs training size.
Results:

- training accuracy decreases as data increases
- validation accuracy improves
- gap reduces → better generalization

Softmax:

	Training Size	Train Accuracy	Validation Accuracy
0	1000	1.00000	0.863917
1	3000	0.96900	0.892417
2	6000	0.94700	0.903250
3	10000	0.93490	0.907583
4	20000	0.92710	0.912667
5	40000	0.92195	0.914167

...



10. Models Compared

Model	Feature	Improvement
Softmax	Raw	Regularization + CV
SVM	Raw	CV
Random Forest	Raw	Ensemble
SVM	CNN	Feature Engineering

Key Findings

- Softmax provides baseline performance
- SVM improves classification margin
- Random Forest captures non-linearity
- CNN + SVM achieves best results

11. Model Comparison Discussion

The comparison of models reveals several important insights.

1. Softmax Regression

- performs well for linear separability
- limited by inability to capture complex patterns.

2. SVM

- better than Softmax due to margin maximization
- handles multiclass classification effectively

3. Random Forest

- captures non-linear relationships
- robust but less effective for high-dimensional pixel data

4. CNN + SVM

This model achieved the best performance because:

- CNN extracts high-level features
- SVM classifies using strong decision boundaries

This combination leverages:

- deep learning for representation
- classical ML for classification

	Model	Feature	Improvement	Accuracy	Precision	Recall	F1-score
0	Linear SVM	CNN Features	CNN Feature Extraction	0.939667	0.939640	0.939048	0.939292
1	Softmax Regression	Raw Pixels	Regularization + CV	0.917250	0.916696	0.916277	0.916343
2	Linear SVM	Raw Pixels	Cross-Validation	0.906667	0.906198	0.905553	0.905632
3	Random Forest	Raw Pixels	Ensemble	0.834000	0.834998	0.830788	0.830553

12.Final Model Selection

Based on validation F1-score:

Best Model = CNN Features + SVM.

13.Final Testing

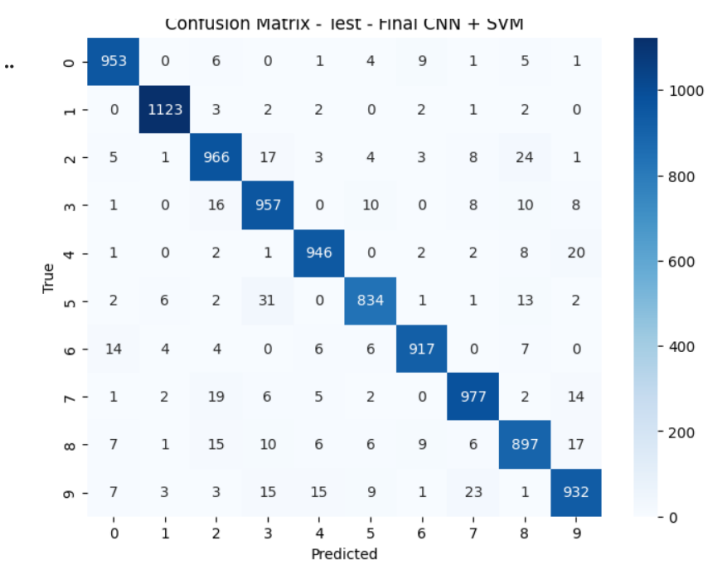
The model was retrained on:

Train + Validation

Then evaluated on the test set.

This ensures:

- no overfitting
- unbiased performance



14. Conclusion

This project successfully demonstrated a complete machine learning pipeline for multiclass classification.

Key achievements:

- implementation of models from scratch
- application of advanced improvement strategies
- analysis of model performance
- identification of best-performing approach

The results show that:

Feature engineering has the greatest impact on performance

The combination of CNN feature extraction and SVM achieved the highest accuracy, demonstrating the effectiveness of combining deep learning representations with classical machine learning models.